

Complemento Práctico: Algoritmos Fundamentales con Colecciones

En la actividad anterior aprendimos a guardar objetos en una lista. Pero guardar datos no sirve de nada si no podemos **extraer información** de ellos. En esta unidad exclusivamente práctica, nos convertiremos en "mineros de datos". Aprenderemos a responder preguntas de negocio como: *"¿Cuál es el promedio de ventas?"*, *"¿Quién es el empleado más antiguo?"* o *"¿Cuánto suman todos los productos?"*.

Para ello, abandonaremos el viejo bucle for con índices (i++) y adoptaremos el elegante **For-Each**. Este cambio no es solo cosmético: representa una forma más segura y expresiva de trabajar con colecciones en Java.



Contenido

El Ciclo For-Each

Sintaxis limpia y moderna para recorrer listas sin errores de índice

Minería de Datos

Patrones de acumulación y cálculo de promedios sobre colecciones

Algoritmo de Extremos

Cómo hallar máximos y mínimos sin errores lógicos comunes

Validaciones Robustas

Evitar división por cero y manejar listas vacías correctamente

La Evolución del Recorrido: For-Each

Del For Tradicional al For-Each

El bucle for tradicional `for (int i=0; i<size; i++)` es propenso a errores: podemos equivocarnos en el índice o salirnos del rango. Para las colecciones, Java ofrece el **For-Each** (Para-Cada), una sintaxis más elegante y segura.

El For-Each se lee de forma natural: *"Para cada Auto (al que llamaremos a) que esté dentro de la lista autos..."*. Esta legibilidad no es accidental, es diseño intencional del lenguaje para hacer el código más expresivo.

Ventaja principal: Es imposible tener un `IndexOutOfBoundsException`. El compilador se encarga de todos los detalles de la iteración por vos.

Comparación: Forma Antigua vs Moderna

Forma Antigua (No Recomendada)

```
// Propenso a errores de índice
for (int i = 0; i < autos.size(); i++) {
    Auto a = autos.get(i);
    System.out.println(a);
}
```

- Riesgo de `IndexOutOfBoundsException`
- Más verboso y difícil de leer
- Requiere gestión manual del índice

Forma Moderna (For-Each)

```
// Sintaxis limpia y segura
for (Auto a : autos) {
    System.out.println(a);
}
```

- Imposible salirse del rango
- Código más limpio y expresivo
- La variable toma cada valor automáticamente

Patrón de Acumulación: Sumatorias y Promedios

Un requerimiento común en programación es sumar valores de objetos almacenados en una colección. Este patrón se llama **acumulación** y es fundamental en el procesamiento de datos. Ya sea que estés calculando totales de ventas, sumando inventarios o promediando calificaciones, el patrón es siempre el mismo.

01

Crear un Acumulador

Declarar una variable externa al ciclo, iniciada en 0 (o el valor neutro apropiado)

02

Recorrer la Colección

Usar For-Each para iterar sobre todos los elementos de la lista

03

Acumular en Cada Vuelta

Sumar el atributo deseado del objeto actual al acumulador

04

Retornar el Resultado

Después del ciclo, devolver el valor acumulado o calcular el promedio



Ejemplo Práctico: Promedio de Kilómetros

Implementación Completa

```
public double
calcularPromedioKm() {
    double sumaKm = 0; //
    Acumulador

    // Validación vital: Evitar
    división por cero
    if (this.autos.isEmpty()) {
        return 0.0;
    }

    for (Auto a : this.autos) {
        sumaKm +=
a.getKilometros();
    }

    return sumaKm /
this.autos.size();
}
```

Validación Crítica

Sin el `if (isEmpty())`, si la lista está vacía, el programa lanzaría **ArithmeticException** o retornaría **NaN** (Not a Number).

Esta verificación es obligatoria en cualquier algoritmo que divida por el tamaño de la colección.

Orden de Operaciones

1. Validar antes de procesar
2. Acumular durante el recorrido
3. Dividir después del ciclo

Búsqueda de Extremos: El Error de Novato

Queremos encontrar el auto con **mayor kilometraje**. Parece simple, pero hay una trampa común que atrapa a casi todos los principiantes: inicializar el "campeón" en cero.

✗ El Error Común

```
int maximo = 0;
```

Problemas:

- ¿Qué pasa si todos los valores son negativos?
- Si buscamos el mínimo con `min = 0`, ¡ganará el 0 aunque no esté en la lista!
- Inventamos un valor que contamina la lógica

✓ La Estrategia Correcta

Asumimos que el primer elemento de la lista es el campeón temporal.

Luego, lo retamos contra los demás. Si alguien le gana, ese se convierte en el nuevo campeón. Nunca inventamos valores, siempre usamos datos reales de la colección.

Este principio aplica a cualquier búsqueda de extremos: máximos, mínimos, más antiguo, más reciente, más caro, más barato. Siempre comenzá con el primer elemento real como referencia.

Implementación Robusta de Máximos

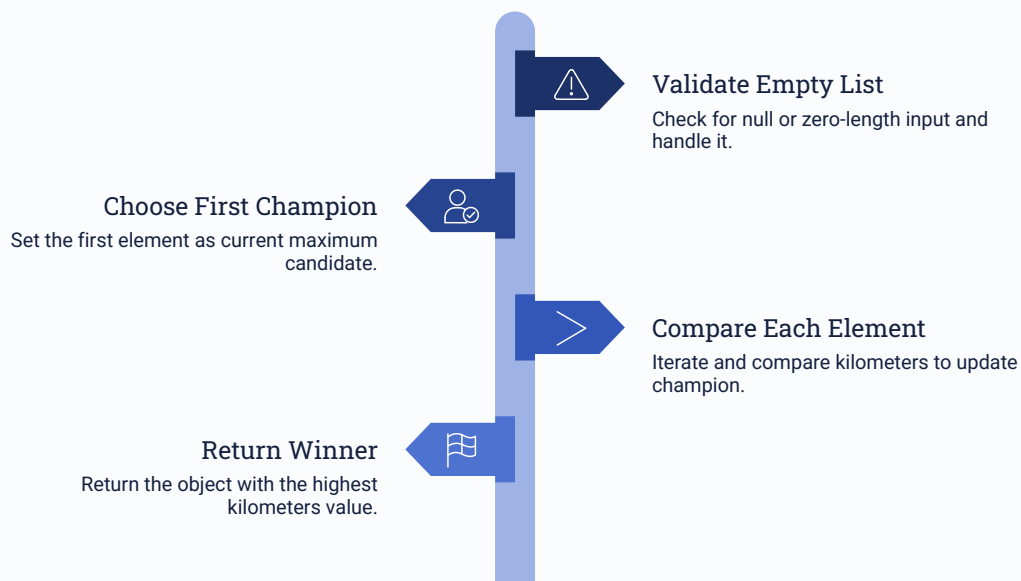
Implementemos la lógica completa para devolver el **objeto Auto** con más kilómetros, no solo el número. Esta implementación sigue las mejores prácticas profesionales de validación y comparación.

```
public Auto buscarAutoMasUsado() {
    // 1. Validar lista vacía
    if (this.autos.isEmpty()) {
        return null;
    }

    // 2. Elegir al primer candidato como campeón
    Auto maxAuto = this.autos.get(0);

    // 3. Recorrer la lista completa
    for (Auto a : this.autos) {
        // 4. El Reto: ¿El auto actual supera al campeón?
        if (a.getKilometros() > maxAuto.getKilometros()) {
            maxAuto = a; // ¡Tenemos nuevo campeón!
        }
    }

    return maxAuto;
}
```



Notá que recorremos toda la lista, incluso el primer elemento. Aunque ya es nuestro campeón inicial, esta redundancia hace el código más uniforme y fácil de mantener.

Tabla Comparativa de Algoritmos

Resumimos los tres algoritmos fundamentales que aprendimos en esta unidad. Cada uno tiene su estructura específica, pero todos comparten el patrón de validación inicial.

Algoritmo	Variable Inicial	Operación en Ciclo	Retorno
Sumatoria	suma = 0	suma += valor	suma
Promedio	suma = 0	suma += valor	suma / size()
Máximo	max = get(0)	if (valor > max) max = valor	max
Mínimo	min = get(0)	if (valor < min) min = valor	min

100%

Validación Obligatoria

Todos los algoritmos deben verificar isEmpty() antes de procesar

0

Valores Inventados

Nunca inicializar extremos con valores arbitrarios como 0 o 999999

1

Patrón Universal

Estos algoritmos funcionan igual en cualquier lenguaje de programación

Resumen: Dominando los Algoritmos sobre Colecciones



En esta actividad hemos dominado la lógica básica sobre estructuras de datos, construyendo una base sólida para el procesamiento de información en cualquier aplicación real.

1 Recorrer con Seguridad

Usamos `for` (Tipo `obj : lista`) por legibilidad y para eliminar errores de índice

2 Acumular Correctamente

Inicializamos variables externas (`suma = 0`) antes del ciclo y las actualizamos dentro

3 Promediar sin Errores

Siempre protegemos la división verificando `.isEmpty()` antes de calcular

4 Extremos con Datos Reales

Nunca inventamos valores iniciales. Siempre tomamos el **primer elemento real** de la lista como referencia

Estos algoritmos son universales y los usarás en cualquier lenguaje de programación. Son las herramientas fundamentales del "minero de datos" profesional. Practicá estos patrones hasta que se vuelvan automáticos, porque son la base de algoritmos más complejos que veremos en unidades futuras.